

Práctica 8 – Implementación de DDD con MVC y ORM

1. Objetivo

Organizar la estructura de directorios/archivos del proyecto de una aplicación web (WebApp o REST) implementando DDD (presentación, servicios de aplicación, dominio e infraestructura – repositorio) con *frameworks* MVC y ORM.

Prerrequisito:

- Equipos de Trabajo de 3 a 6 miembros.
- Laboratorio 6 y 7
- Lenguaje de Programación: Java, C# o Python
- **IDE:** Eclipse, IntelliJ IDEA, VS Code MS Visual Studio, ...
- **Framework Web MVC:** Springboot (Java), ASP.NET MVC (C#) o Flask/Django (Python)
- **Framework ORM:** JPA o Hibernate (Java), Entity Framework (C#) o SQLAlchemy (Python). En caso de MongoDB, usar un **ODM**
(<https://www.mongodb.com/developer/products/mongodb/mongodb-orms-odms-libraries>)

2. Actividades:

EL EQUIPO DEBE:

- 1) Actualizar el **Modelo de Dominio y Arquitectura:**
 - a) Especificar **operaciones CRUD** (crear, buscar, actualizar y eliminar elementos del dominio) a ser implementadas en los **repositorios** concretos.
 - b) Especificar **operaciones desde el punto de vista del usuario final** (funcionalidades como **casos de uso**, **features BDD** o **historias de usuario**) que su dominio expone a través de los **servicios de aplicación** y **controladores**.
 - c) Nombrar Paquetes, Clases e Interfaces de acuerdo a las Convenciones/Estándares de Nombrado (**Naming Conventions**) del lenguaje de programación (**Java, C# o Python**) escogido.
 - d) Organizar la estructura del proyecto (capas/paquetes/directorios, archivos y clases) de acuerdo a las reglas impuestas por los **Frameworks MVC (Spring, ASP.NET MVC o Flask/Django)** y **ORM (JPA, Entity Framework o SQLAlchemy)** seleccionados para el desarrollo del proyecto.
- 2) (Re)Generar la **ESTRUCTURA DEL PROYECTO:** Directorios y archivos de **código fuente** del proyecto a partir de la "Arquitectura de Software".

En StarUML:

- a) Limpiar el modelo de arquitectura de tal forma que este represente fielmente la estructura del proyecto: Paquetes + Clases
- b) Nombrar el Modelo de Acuerdo al Proyecto.
- c) Generar el **código fuente del proyecto** a partir del modelo de arquitectura, por medio de una extensión (Java, C# o Python) del lenguaje de programación elegido.
- d) Verificar la consistencia entre el código fuente generado y el modelo base.

En su IDE favorito:

- a) Crear un **proyecto** web.
- b) Importar las **dependencias** necesarias (*frameworks* MVC y ORM) al proyecto generado.
- c) Importar el **código fuente del proyecto** generado por **StarUML**.
- d) Configurar el **proyecto** de acuerdo a las restricciones del lenguaje de programación y **frameworks** MVC y ORM, caso sea necesario.
- e) Verificar:

- **Nombre** de Paquetes, Módulos, Clases, Interfaces, Atributos/Argumentos y Operaciones de acuerdo a las **Convenciones/Estándares de Nombrado** (*Naming Conventions*) del lenguaje de programación (Java, C# o Python) escogido.
 - **Estructura** de Paquetes y nombres de directorios/archivos respetando las directrices de los **Frameworks MVC** (Spring, ASP.NET MVC o Flask/Django) y **ORM** (JPA, Entity Framework o SQLAlchemy) seleccionados: presentación (vista y controlador); aplicación (servicio); dominio (modelo organizado en módulos); infraestructura (repositorio, acceso a servicios externos).
- f) Implementar la página de inicio o bienvenida de la aplicación web: versión inicial incompleta.
- g) Construir la aplicación: compilar
- h) Ejecutar la aplicación

3) Crear un repositorio en **GitHub** para su proyecto:

El líder del equipo debe:

- a. Crear un **repositorio** del equipo en GitHub: Nombre del proyecto.
- b. Crear una **rama** (branch) de desarrollo, además de la **rama** master.
- c. Subir (commit + push) el proyecto al **repositorio GitHub** del equipo (ramas *master* y desarrollo): **commit** y **push** con nombres descriptivos:
 - Buenas prácticas en **commits**: <https://midu.dev/buenas-practicas-escribir-commits-git/> <https://www.freecodecamp.org/espanol/news/como-escribir-un-buen-mensaje-de-commit/>

Cada miembro del equipo debe tener una **copia** del repositorio **GitHub** del equipo:

- a. Crear cuenta GitHub
 - b. Hacer **Clone** o **Fork** al repositorio del equipo: rama desarrollo.
 - c. Configurar el proyecto en su entorno local de trabajo: IDE.
 - d. Subir (*Upload* o *Commit + Push* o *Pull Request*) sus cambios a la rama de desarrollo, cuando sea necesario.
 - e. Solicitar al propietario del repositorio del equipo para que este último incorpore los cambios (vía *Merge* o *Pull Request*), cuando sea necesario.
- 4) Distribuir esfuerzos y tareas entre los miembros del equipo por **Trello**.
- a. El Líder del equipo debe: Asignar responsables y plazos para la implementación de las historias de usuario del proyecto.
 - b. Los miembros del equipo deben:
 - Crear **Checklists** para definir tareas de implementación (Por ejemplo: crear GUI, diseñar esquema de BD, implementar consultas por ORM, estudiar *framework* X, ...) correspondientes a las historias de usuario asignadas.
 - Poner las historias de usuario en ejecución
 - Realizar pruebas
 - c. El Líder del equipo debe: hacer seguimiento a la implementación de las historias de usuario del proyecto.
- 5) Actualizar el avance del proyecto por **Trello**:
- a) Iteración actual:
 - Etapas: mover **tarjetas** que están siendo implementadas,
 - Responsables: Por implementar, y plazos,
 - Features:
 - **Checklist**: ~~"Escenarios de Prueba": adicionar los <escenarios> a ser implementados y probados.~~
 - **Epics** o Historias asociadas a una *feature* de la iteración actual:
 - **Checklist**: "Tareas de Implementación": adicionar tareas de implementación, por ejemplo, como "Implementar Controlador (REST); Estudiar Java y JPA y Hibernate; Mapear Dominio con ORM; Implementar Repositorio, ...

3. Recursos de Software:

- <https://trello.com>
- <https://staruml.io>
- <https://github.com/>

Entregable:

- Repositorio **GitHub** del equipo [cuya estructura debe estar organizada de acuerdo a las recomendaciones de *DDD*, el lenguaje de programación y los *frameworks* elegidos. El README debe presentar la siguiente información:
 - Propósito
 - Funcionalidades: Funcionalidades de Alto Nivel (Diagrama de Casos de Uso UML) y Prototipo (o GUI)
 - Modelo de Dominio: Diagrama de Clases + Módulos.
 - Vista General de Arquitectura: Diagrama de Paquetes + Clases
- Tablero **Kanban/Scrum** de Proyecto basado en la plantilla "**User Story mapping**" en **Trello** u otra herramienta similar.

Referencias

- E. Evans. Domain-Driven Design: Tackling Complexity in the Heart of Software. 2003
- Valente, M. T. (2020). Engenharia de Software Moderna (Livro Digital). Disponible em: <https://softengbook.org/>
- Vernon, V. (2016). Domain-driven design distilled. Addison-Wesley Professional.
- Marinescu, F., & Avram, A. (2007). Domain-driven design Quickly. <https://www.infoq.com/minibooks/domain-driven-design-quickly/>
- Valente, M. T. (2020). [Engenharia de Software Moderna](#) (Livro Digital).
- M Richards. 2015. [Software architecture patterns](#).
- Mark Richards & Neal Ford. (2023). Fundamentals of software architecture: an engineering approach. O'Reilly.
- Garlan, David, and Mary Shaw. "An introduction to software architecture." Advances in software engineering and knowledge engineering. 1993. 1-39.
- Modelo de Dominio:
 - Demián Gutierrez. Universidad de Los Andes. UML Diagramas de Clases. Disponible en: http://www.codecompiling.net/files/slides/UML_clase_04_UML_clases.pdf
 - What are the six types of relationships in UML class diagrams?. Disponible en: <https://blog.visual-paradigm.com/what-are-the-six-types-of-relationships-in-uml-class-diagrams>
 - UML Class Diagram Tutorial. Disponible en: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial>

- Arquitectura:
 - Patterns of Modular Architecture: <https://dzone.com/refcardz/patterns-modular-architecture#section-4>
 - Modularity in Java and .Net: <https://dev.to/pathiknd/modularity-in-java-and-net-3dnj?ref=hackernoon.com>
 - Modularity in Java: <https://www.baeldung.com/java-modularity>
<https://www.upgrad.com/blog/modularity-in-java/>
 - Modularity in .Net: <https://codewithmukesh.com/blog/modular-architecture-in-aspnet-core/>
 - Modularity in Python: <https://plainenglish.io/blog/modularization-on-python>
<https://docs.python.org/3/tutorial/modules.html>

Material de Aula

- Convenciones del Lenguaje de Programación:
 - **JAVA:** <https://www.oracle.com/technetwork/java/codeconventions-150003.pdf>
 - **C#:** <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names>
 - <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>
 - **Python:** <https://peps.python.org/pep-0008/#naming-conventions>
 - <https://realpython.com/python-pep8/#naming-styles>
 - **Principios KISS, DRY y YAGNI:** <https://medium.com/@hlfddev/kiss-dry-solid-yagni-a-simple-guide-to-some-principles-of-software-engineering-and-clean-code-05e60233c79f>
- Guías de Implementación de DDD y CLEAN ARCHITECTURE CON MVC y ORM (Ver diapos. DDD y otras fuentes):
 - **Java:** <https://www.baeldung.com/spring-boot-start>
 - <https://www.baeldung.com/hexagonal-architecture-ddd-spring>
 - <https://www.digitalocean.com/community/tutorials/spring-mvc-hibernate-mysql-integration-crud-example-tutorial>
 - <https://www.geeksforgeeks.org/spring-mvc-and-hibernate-crud-example/>
 - **C#:** <https://learn.microsoft.com/en-us/aspnet/mvc/overview/getting-started/>
 - <https://cantinhode.net/blogs/community-cantinho-de-net/implementing-domain-driven-design-in-asp-net-mvc>
 - <https://banisd.wordpress.com/software-thoughts/ddd-mvc-entity-framework/>
 - **Python:** <https://realpython.com/flask-connexion-rest-api/>
 - <https://realpython.com/flask-project/>
 - <https://flask.palletsprojects.com/en/3.0.x/>
 - <https://www.digitalocean.com/community/tutorials/how-to-structure-a-large-flask-application-with-flask-blueprints-and-flask-sqlalchemy>